

Aufgabe 11

Aufgabe 11.1

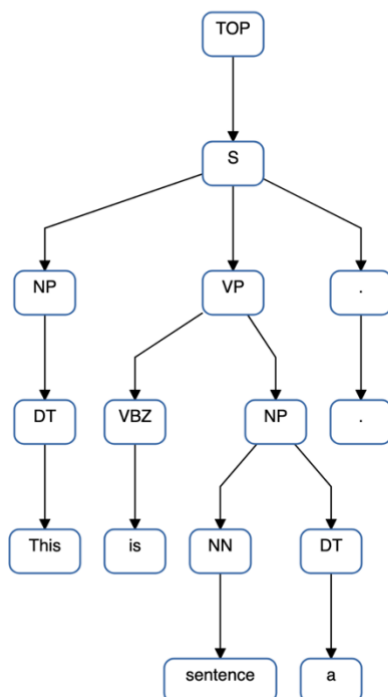
Erklärung:

Der visualisierte Satz „This is a sentence.“ enthält am Ende ein Satzzeichen, das vom Parser als eigenständiges Token behandelt wird – genau wie jedes Wort. Deshalb bekommt der Punkt im Parsebaum zwei eigene Knoten: einen Wort-Knoten mit dem Token „.“ und darüber einen POS-Tag-Knoten, der in der Penn-Treebank-Konvention ebenfalls „.“ heißt (das ist dort der offizielle Tag für Satzendzeichen, analog zu DT für Determiner oder NN für Noun). Da Label und Tag in diesem Sonderfall identisch sind, erscheint in beiden Knoten ein Punkt statt eines Buchstabens.

Screenshot:

Übung 11.1 – Sentence Tokenization mit Dagre

Satz: "This is a sentence." – Layout via Dagre (Sugiyama-Methode), Pfeile zeigen die Kantenrichtung.



Link: <https://ellie-app.com/yTrpG2TH9tga1>

Aufgabe 11.2

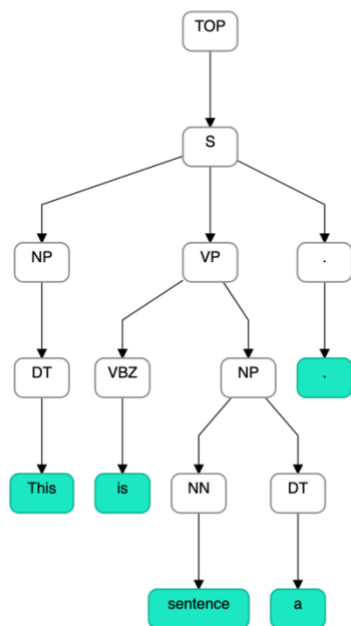
Erklärung:

Der Elm-Code aus 11.1 wurde erweitert, sodass jeder Knoten zusätzlich zum Label eine CSS-Klasse trägt: "leaf" für Wort-Tokens (This, is, a, sentence, .) und "inner" für POS-Tags und Phrasen-Knoten (S, NP, VP, DT usw.). Diese Klasse wird mit in das JSON geschrieben, das an Dagre geschickt wird, beim Layout-Ergebnis wieder dekodiert und schließlich beim Zeichnen über `TypedSvg.Attributes.class` an jedes `<g>`-Element gehängt (`class="node leaf"` bzw. `class="node inner"`). Im HTML-Wrapper werden dann CSS-Regeln (`g.node.leaf rect { fill: #1ae8c3 ... }`, `g.node.inner rect { fill: #fff ... }`) diese Klassen aus und färben die Wort-Knoten grün, während POS-/Phrasen-Knoten weiß bleiben – analog zum Referenzbild der Vorlesung.

Screenshot:

Übung 11.2 – Sentence Tokenization mit CSS-Klassen

Satz: "This is a sentence." – Layout via Dagre, Wort-Tokens (Klasse "leaf") grün, POS-/Phrasen-Knoten (Klasse "inner") weiß.



Link: <https://ellie-app.com/yTrN6ZSrb2wa1>

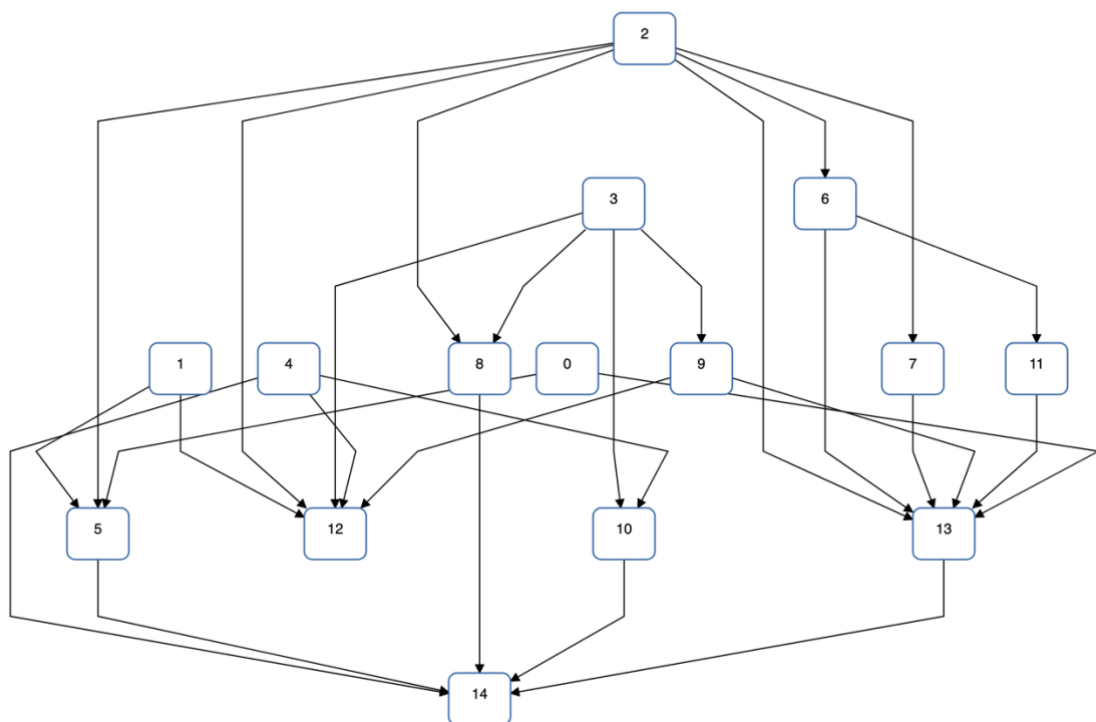
Aufgabe 11.3

Erklärung:

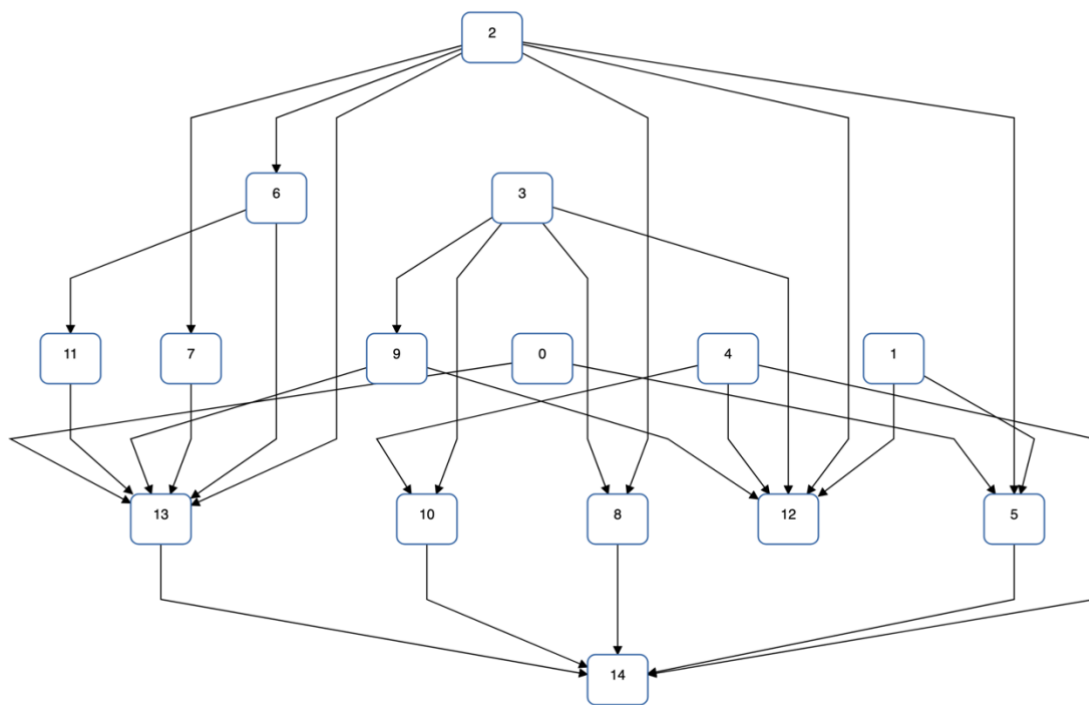
Der DAG aus Folie 456 (15 Knoten 0–14, 27 Kanten) wird einmal in Elm als `testGraph` definiert und beim `init` in drei JSON-Varianten an den `layout`-Port geschickt – die einzige Differenz ist das Feld `value.ranker` mit den Werten "network-simplex", "tight-tree" und "longest-path" (die drei von Dagre unterstützten Layering-Algorithmen). Im HTML-Wrapper iteriert das JavaScript über das Array, ruft für jeden Graphen `dagre.layout(...)` auf und sendet die drei Ergebnisse als Liste zurück über den `graphs`-Port. Im `update` werden sie in drei Maybe GraphData-Felder einsortiert (`networkSimplex`, `tightTree`, `longestPath`) und im `view` mit identischer SVG-Render-Funktion untereinander gezeichnet, sodass man die Unterschiede der drei Ranker direkt vergleichen kann.

Screenshots:

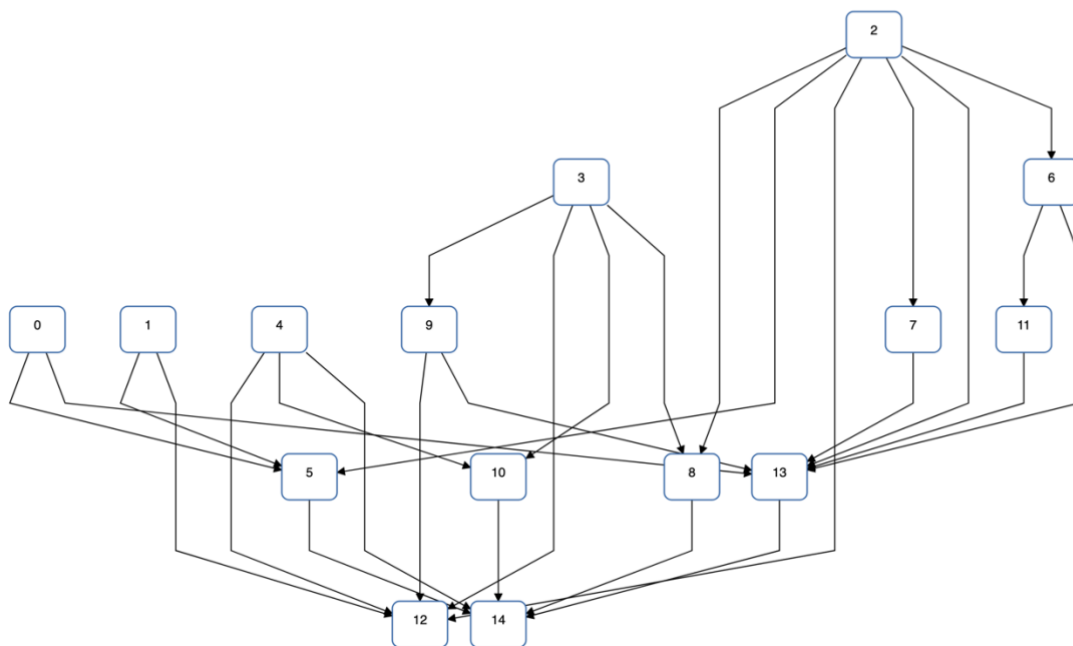
a) ranker = network-simplex



b) ranker = tight-tree



c) ranker = longest-path



Link: <https://ellie-app.com/yTs2xL7hhMva1>